



Identification multi-level frequent usage patterns from APIs



Hamzeh Eyal Salman ^{a,b,*}

^a Mutah University, Karak, Jordan

^b LIRMM, University of Montpellier 2, Montpellier, France

ARTICLE INFO

Article history:

Received 5 October 2016

Revised 13 March 2017

Accepted 12 May 2017

Available online 15 May 2017

Keywords:

API documentation

API usage

Formal concept analysis

Identification

Usage patterns

ABSTRACT

Software developers increasingly rely on application programming interfaces (APIs) of frameworks to increase productivity. An API method is generally used within code snippets along with other methods of the API of interest. When developers invoke API methods in a framework, they often encounter difficulty to determine which methods to call due to the huge number of included methods in that API. Developers usually exploit a source code search tool searching for code snippets that use the API methods of interest. However, the number of returned code snippets is very large which hinders the developer to locate useful ones. Moreover, co-usage relationships between API methods are often not documented. This article presents an approach to identify multi-level frequent usage patterns (IML-FUP) to help developers understand API usage and facilitate the development tasks when they use new APIs. An identified pattern represents a set of API methods that are frequently called together across interfering usage scenarios. In order to investigate the efficiency of the proposed approach, an experimental evaluation is conducted using four APIs and 89 client programs. For all studied APIs, the experimental results show that the proposed approach identifies usage patterns that are always strongly cohesive and highly consistent.

© 2017 Elsevier Inc. All rights reserved.

1. Introduction

Today, development software applications increasingly relies on application programming interfaces (APIs). They are considered as the most commonly used repositories for code reuse (Shatnawi et al., 2016). APIs provide a set of pre-implemented, pre-tested functionalities that can be reused instead of developing them from scratch. Hence, they increase software quality and reduce the effort consuming in programming, testing and maintenance activities.

Despite the research work undertaken in documenting and understanding APIs (Saied et al., 2015b; Uddin et al., 2012; Wang et al., 2013), most of the APIs are complex and hard to learn and use (Hou and Li, 2011; Wang and Godfrey, 2013). Generally, an API method is used along with other methods of the API of interest, and thus programmers spend considerable time and effort to pick out API's methods that are always or frequently used together and at the same time necessary to accomplish their daily work. These difficulties are due to three reasons. Firstly, the relationships between API methods (called co-usage relationships) are often not documented. This is because the documentation of an API method is often limited to the functionality implemented by that method alone and without any details about other API's methods that usu-

ally are used together with that method. Secondly, the documentation of an API class may be very detailed and elaborated, verbose text, as it tries to cover all aspects that a programmer might need to know about the usage of classes of interest. Finally, APIs often provides a large number of classes and methods.

Programmers can learn how to use API's methods through searching for code snippets using a source code search engine such Krugle code search engine.¹ For a given API method, existing source code search engines return a huge number of code snippets that use the API's method of interest. Browsing all code snippets found to locate the proper snippet is practically impossible. Therefore, the identification of API usage patterns from a huge number of code snippets is useful to help programmers how to use API's methods.

In recent years, a few techniques were proposed to identify API usage patterns for the purpose documentation and understanding (Robillard et al., 2013; Uddin et al., 2012; Wang et al., 2013; Zhong et al., 2009; Saied et al., 2015b). In their techniques, a usage pattern is defined as a set of API elements, classes or methods, that are used together. Most of these techniques focus on identifying such patterns in a specific usage scenario and do not reflect interference between different API usage scenarios. This interference could negatively impact the process of API usage patterns identification (Saied et al., 2015b). Such techniques are proposed to pro-

* Corresponding author.

E-mail addresses: hamzehmu@mutah.edu.jo, hamzehahu@gmail.com

¹ <http://opensearch.krugle.org/document/search/#query=setproxy&language=Java>.

vide recommending usage examples of API's methods that are relevant to the task at hand (e.g., Zhong et al., 2009). Moreover, all existing techniques are semi-automatic as they need user intervention during the identification process. As a consequence of above-mentioned limitations, they fail automatically identifying API usage patterns that are characterized as both *context-independent* and *reflect interference between different API usage scenarios*.

In this article, an approach (called **Identification Multi-Level Frequent Usage Pattern**, for short IML-FUP) to automate the identification of usage patterns from given API is proposed. IML-FUP identifies a pattern as multiple levels of usage cohesion of API's methods. The rationale behind this multiple levels organization of a pattern is to overcome the limitations of existing work. On the one hand, to identify the pattern's core, which represents a set of API methods that 'always' are used together regardless of the context used (context-independent). On the other hand, to reflect interference between API usage scenarios through identifying API's methods that 'frequently' are used along with the pattern's core and their usage depending on usage scenario. To identify such patterns, the co-usage relationships between methods of API of interest are analyzed in terms of cohesion and consistency in a variety of client programs.

In order to evaluate the IML-FUP approach, experiments have been conducted using four APIs with 89 client programs from different domains. These APIs are *HttpClient*,² *Java Security*,³ *Swing*⁴ and *AWT*.⁵ In our evaluation, 30 clients were used for AWT and Swing, 17 clients were used for Java Security and 12 clients were used for HttpClient. The experimental results show that, for every API studied, usage patterns identified by the IML-FUP approach are always strongly cohesive. In order to evaluate the consistency (generalization) of identified usage patterns, leave-one-out cross validations are performed across new client programs offering completely different features. The results show that the identified patterns from each studied API are highly consistent across a considerable variability of client programs.

The remainder of this article is organized as follows: Section 2 provides a necessary background to understand the approach. Section 3 details the proposed approach steps. Next, Section 5 shows and discusses experimental results, together with the threats to validity. Then, Section 6 discusses the related work. Finally, Section 7 concludes the article and indicates to the future work.

2. Background and motivation examples

2.1. Formal concept analysis

Formal Concept Analysis (FCA) is a technique for data analysis and knowledge representation based on lattice theory (Ganter and Wille, 1999; Barbut and Monjardet, 1970; Berry et al., 2014). It identifies meaningful groups of objects sharing common attributes and provides a theoretical model to analyze hierarchies of these groups. This technique is currently applied to support various tasks, including data mining (Valtchev et al., 2004), building or maintaining class hierarchies in object-oriented software (Godin and Mili, 1998), software understanding (Bhatti et al., 2012; Poshyanyk and Marcus, 2007).

To better understand FCA, it is presented along with an illustrative example. Suppose a list of Mexican dishes and a list of ingredients for each dish. Assume the data set in Table 1 is for these

Table 1
Mexican dishes and their ingredients.

Mexican dish	Ingredients
Burritos	chicken, beef, pork, vegetables, beans, rice, cheese, guacamole, sour-cream, lettuce and flour-tortilla
Enchiladas	chicken, cheese, sour-cream and corn-tortilla
Fajitas	chicken, beef, vegetables, cheese, guacamole, sour-cream, lettuce and flour-tortilla
Nachos	vegetables, beans, cheese and guacamole
Quesadillas	chicken, beef, cheese, corn-tortilla and flour-tortilla
Tacos	chicken, beef, beans, cheese, lettuce, corn-tortilla and flour-tortilla

Mexican dishes. Rows represent dishes while the columns represent the ingredients of each dish.

Definition 1 (Formal Context). A formal context is defined as a triple $K = (O, A, R)$ where O is a set of objects, A is a set of attributes and R is a binary relation between objects and attributes indicating which attributes are possessed by each object, i.e., $R \subseteq O \times A$.

A formal context is represented as a cross table in which row labels display objects and column labels display attributes. Table 2 shows a formal context for Mexican dishes and their ingredients. A cross in the cell (o, a) of this table refers to the object o that possess the attribute a . The key modeling issue that strongly impacts on the analysis performed by FCA is the right choosing of objects, attributes and relations. From our illustrative example, we can create a formal context of Mexican dishes $O = \{\text{Burritos}, \text{Enchiladas}, \text{Fajitas}, \text{Nachos}, \text{Quesadillas}, \text{Tacos}\}$ and their ingredients $A = \{\text{chicken}, \text{beef}, \text{pork}, \text{vegetables}, \text{beans}, \text{rice}, \text{cheese}, \text{guacamole}, \text{sour-cream}, \text{lettuce}, \text{corn-tortilla}, \text{flour-tortilla}\}$.

For a set of objects $M \subseteq O$, then $M' = \{a \in A \mid \forall o \in M : (o, a) \in R\}$ is the set of common attributes. Also, for a set of attributes $B \subseteq A$, then $B' = \{o \in O \mid \forall a \in B : (o, a) \in R\}$ is the set of common objects. For example, if we take the set $M = \{\text{Enchiladas}, \text{Quesadillas}, \text{Tacos}\}$ from Table 2, the set of common attributes is $M' = \{\text{chicken}, \text{cheese}, \text{corn-tortilla}\}$. In the same way, if $B = \{\text{pork}, \text{rice}\}$ then, $B' = \{\text{Burritos}\}$.

Definition 2 (Formal Concept). For a given formal context $K = (O, A, R)$, a formal concept is a pair (E, I) composed of an object set $E \subseteq O$ and an attribute set $I \subseteq A$. $E = \{o \in O \mid \forall a \in I, (o, a) \in R\}$ is the extent of the concept (i.e., all objects covered by the concept). $I = \{a \in A \mid \forall o \in E, (o, a) \in R\}$ is the intent of the concept (i.e., all attributes shared by the objects covered by the concept).

For example, $(\{\text{Enchiladas}, \text{Quesadillas}, \text{Tacos}\}, \{\text{chicken}, \text{cheese}, \text{corn-tortilla}\})$ is a concept, while $(\{\text{Nachos}\}, \{\text{vegetables}, \text{beans}, \text{cheese}, \text{guacamole}\})$ is not, because $(\{\text{Nachos}\})' = \{\text{vegetables}, \text{beans}, \text{cheese}, \text{guacamole}\}$ while $(\{\text{vegetables}, \text{beans}, \text{cheese}, \text{guacamole}\})' = \{\text{Burritos}, \text{Nachos}\}$.

Definition 3 (Concept Specialization Order). Given a formal context $K = (O, A, R)$, and two formal concepts $C_1 = (E_1, I_1)$ and $C_2 = (E_2, I_2)$ of K , the concept specialization order ($\leq_s$) is defined by $C_1 = (E_1, I_1) \leq_s C_2 = (E_2, I_2)$ if and only if $E_1 \subseteq E_2$ (and equivalently $I_2 \subseteq I_1$). C_1 is called a sub-concept of C_2 . C_2 is called a super-concept of C_1 .

For example, $(\{\text{Burritos}\}, \{\text{chicken}, \text{beef}, \text{pork}, \text{vegetables}, \text{beans}, \text{rice}, \text{cheese}, \text{guacamole}, \text{sour-cream}, \text{lettuce}, \text{flour-tortilla}\})$ is a sub-concept of $(\{\text{Burritos}, \text{Nachos}\}, \{\text{vegetables}, \text{beans}, \text{cheese}, \text{guacamole}\})$. Due to this specialization order definition, an evident property is that a sub-concept owns (inherits in top-down manner) the attributes of its super-concepts, while a super-concept owns (inherits in bottom-up manner) the objects of its sub-concepts.

² <http://hc.apache.org/httpclient-3.x/>.

³ <http://docs.oracle.com/javase/7/docs/api/java/security/package-summary.html>.

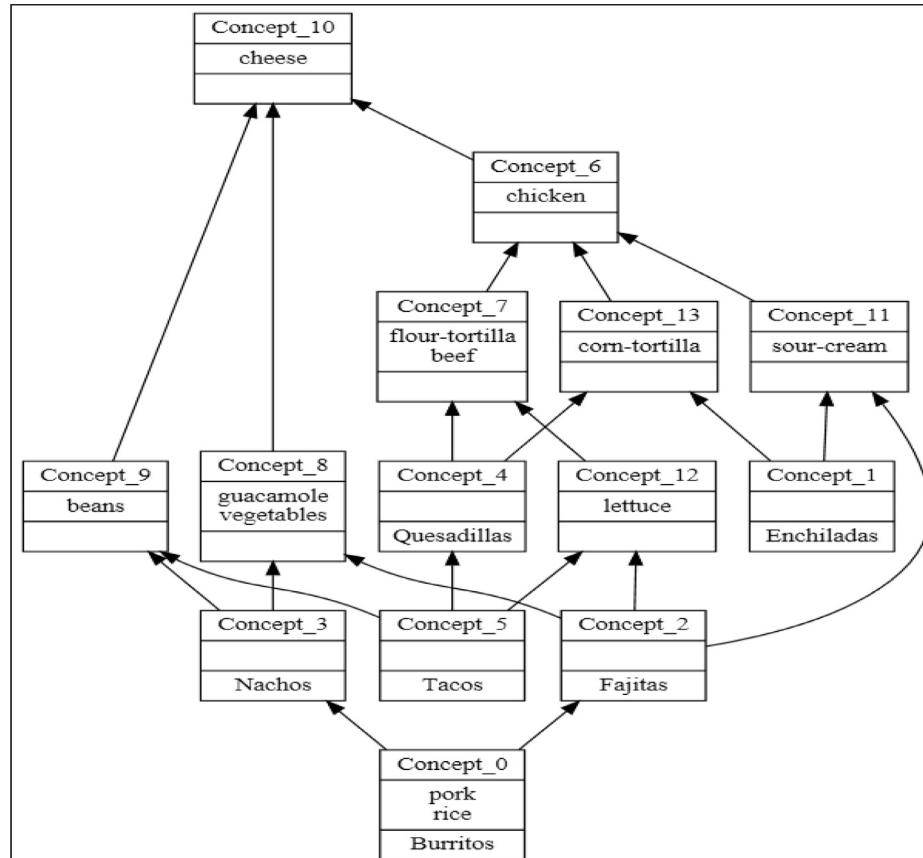
⁴ <http://docs.oracle.com/javase/7/docs/technotes/guides/swing/>.

⁵ <http://docs.oracle.com/javase/7/docs/api/>.

Table 2

A formal context for Mexican dishes.

	chicken	beef	pork	vegetables	beans	rice	cheese	guacamole	sour-cream	lettuce	corn-tortilla	flour-tortilla
Burritos	X	X	X	X	X	X	X	X	X	X		X
Enchiladas	X						X		X		X	
Fajitas	X	X		X			X	X	X	X		X
Nachos				X	X		X	X				
Quesadillas	X	X					X				X	X
Tacos	X	X			X		X			X	X	X

**Fig. 1.** GSH-Lattice for the formal context in Table 2.

Definition 4 (Concept Lattice). Let C_K be the set of all concepts of a formal context K . This set of concepts provided with the specialization order ($C_K, \leq_s$) has a lattice structure, and is called the concept lattice associated with K .

Fig. 1 shows the concept lattice associated with the formal context of Table 2. This lattice is known Galois Sub-Hierarchy (GSH). Such a type of concept lattice has not empty concepts where each concept has at least one object or one attribute. In this lattice we can discover many facts as well as the relationships between the presented Mexican dishes, for example:

- All the Mexican dishes contain *cheese* because it appears as the intent of the top concept (*Concept_10*) that has all the dishes in its extent.
- When a concept owns only Mexican dishes, this means that these dishes don't have specific ingredients and they share ingredients with other dishes belonging to different concepts, for example: in *Concept_3*, *Nachos* inherits *vegetables*, *guacamole*, *beans* and *cheese*.
- When a concept owns only ingredients, this means that these ingredients are not specific for a certain Mexican dish but they are shared among different dishes which belong to multiple

concepts, for example: in *Concept_9*, *beans* is shared between *Nachos*, *Tacos* and *Burritos* dishes.

Most lattice representations show attributes and objects solely where they are introduced (not repeating the inherited ones). They show the simplified intents and simplified extents. Most of the algorithms for building concept lattices are cited and compared in Kuznetsov and Obiedkov (2002).

2.2. Frequent usage patterns

In data mining, frequent usage patterns occur frequently in a dataset (Han, 2005). Generally, a frequent usage pattern (FUP) is a set of items that often appear or are used together in a transactional dataset. For example, milk and bread, which are frequently bought together in grocery stores by many customers.

In this article, a FUP is a group of methods of the API of interest that are co-used together by the API client programs. These methods represent only *public* methods of that API. A FUP consists of two parts: core and peripheral. The core of a usage pattern represents a set of methods that 'always' are called together regardless of usage context. The peripheral part of a usage pattern represents

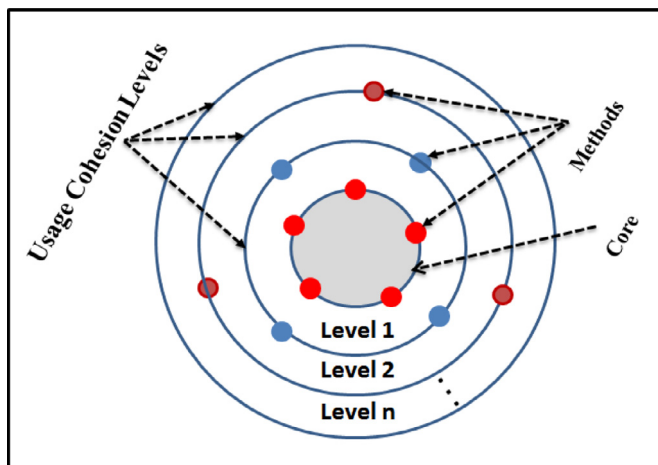


Fig. 2. A graphical representation of a multi-level frequent usage pattern.

a set of methods that ‘frequently’ are used with the core of that pattern. The peripheral methods are distributed into levels according to the degree of usage cohesion with core part. Thus, the peripheral methods are closely associated with core methods, from the closest to the least close ones. Such organization of a pattern constitutes a multi-level frequent usage pattern which includes the core and peripheral parts as shown in Fig. 2.

Such organization of a multi-level frequent usage pattern provides valuable information. For a given pattern, all core’s methods are invoked together regardless of the context. Most of client programs invoke partially or totally, as needed, methods located in the Level 1. Some client programs invoke partially or totally methods located in the Level 2 and so on.

2.3. Motivational examples

In this section, two motivation examples are provided to illustrate the following aspects related to the problem addressed: (1) the same core frequent usage pattern can be reused in different contexts (context-independent usage pattern); (2) the interference between different API usage scenarios could be presented as a multi-level frequent usage pattern.

2.3.1. Authentication in HttpClient API

To better understand the benefit of using the same core frequent usage pattern across different contexts, authentication in HttpClient API is introduced as an example. HttpClient API provides an efficient, up-to-date and feature-rich package implementing the client side of the most recent HTTP standards and recommendations (Jakarta commons httpclient component, 2017). It may be of interest to anyone building HTTP-aware client applications such as web browsers or systems that leverage or extend the HTTP protocol for distributed communication (Jakarta commons httpclient component, 2017).

In HttpClient API, the method `setProxy` (*String proxyHost, int proxyPort*) of *HostConfiguration* class is always invoked together with the method `setProxyCredentials` (*AuthScope authscope, Credentials credentials*) of *HttpState* class (Saied et al., 2015b). This is because by analysis many client programs of HttpClient API found that both methods are always invoked together. In fact, proxy authentication requires credentials (HttpClient, 2017). Therefore, when proxy settings are adjusted in HTTP-client program using the `setProxy()`, the client program should set the proxy credentials for the given authentication realm (domain) using `setProxyCredentials()` method.

Fig. 3 shows two code snippets for *HostConfiguration.setProxy()* method found using Krugle code search engine which returns 444 code snippets.⁶ In both code snippets, the methods `setProxy()` and `setProxyCredentials()` are called together in different client projects. The first code snippet is taken from *GeoTools*⁷ project while the second code snippet is taken from *Apache JMeter*⁸ project. Moreover, the purpose of the client classes/methods of these methods are completely different as their names indicate. In the first code snippet, the methods of interest are called by `applySystemProxySettings` method. In the second code snippet, the methods of interest are called by `setupConnection` class. It is worth noting that by deeply analyzing other code snippets the co-usage relationship between `setProxy()` and `setProxyCredentials()` remain consistent.

It is important to mention that by reviewing the Java documentation⁹ of `setProxy()` and `setProxyCredentials()` and reading the *HttpClient*’s code samples,¹⁰ there is no documentation for the co-usage relationship between above mentioned methods. Moreover by reading the authentication guide section in *HttpClient* user guide,¹¹ the reader finds that the first paragraph describes “Server Authentication” while the fourth paragraph describes “Proxy Authentication”. Both paragraphs do not document the co-usage relationship between the methods of interest.

2.3.2. Interface layout designing in Swing API

To better understand the benefit of identifying consistent usage patterns across a considerable variability of client programs, interface layout designing in Swing API is introduced as an example (Saied et al., 2015b). In Swing API, the class *groupLayout* is a layout manager that hierarchically groups graphical components in order to position them in a container.¹² Usually, this class is used by every client program uses graphical user interface (GUI). This class consists of 30 methods in addition to the constructor *GroupLayout(Container host)*.¹² To find the smallest group of *GroupLayout*’s methods that are actually needed for managing Swing interface layouts, the *GroupLayout*’s Java documentation,¹² the associated examples and the Java Swing tutorial,¹² “How to Use GroupLayout” and “A GroupLayout Example” should be explored to perform that. However, this is not a trivial task. Indeed, the example associated with *groupLayout* documentation uses 6 methods while in Java Swing tutorial the example associated with *groupLayout* uses the same methods in addition to another method.

By analyzing a variety of code snippets that use “*GroupLayout* class”, it was found there are a subset of *GroupLayout*’s methods that are consistently called together (Saied et al., 2015b). These methods are: *GroupLayout()*, *setHorizontalGroup()* and *setVerticalGroup()*. Such methods constitute the core part of frequent usage pattern (see Fig. 4). In addition to these methods, there are two other methods that must be used either one of them or both to complete building a layout. These methods are: *createParallelGroup()* and *createSequentialGroup()*. Such methods constitute the peripheral part of frequent usage pattern peripheral (see Fig. 4).

In summary, the IML-FUP approach can help users of *HttpClient* and *Swing* APIs to find methods that should always be used together (core) and method that used according to the need (peripheral). Hence, the users focus only on those methods instead

⁶ The query “setProxy & language = Java” using Krugle code search engine was performed in June 2016. Note that given the same key words, Krugle code search may returns different numbers of code snippets over time possibly due to the growth of Krugle code search’s repositories.

⁷ GeoTools is an open source Java library that provides tools for geospatial data.

⁸ Apache JMeter is open source software designed to load test web functional behavior and measure performance.

⁹ <https://hc.apache.org/httpclient-3.x/apidocs/>.

¹⁰ <http://svn.apache.org/viewvc/httpcomponents/oac-hc3x/trunk/src/examples/>.

¹¹ <http://hc.apache.org/httpclient-3.x/authentication.html>.

¹² <http://docs.oracle.com/javase/7/docs/api/javax/swing/GroupLayout.html>.

```

public class MultithreadedHttpClient implements HttpClient {
    .....

    private void applySystemProxySettings() {
        .....
        HostConfiguration hostConfig = client.getHostConfiguration();
        hostConfig.setProxy(proxyHost, proxyPort);
        .....

        HttpState state = client.getState();
        .....
        AuthScope authscope = AuthScope.ANY;
        Credentials credentials = new UsernamePasswordCredentials(proxyUser, proxyPassword);
        state.setProxyCredentials(authscope, credentials);
        .....
    }
}

public class HTTPHC3Impl extends HTTPHCAbstractImpl {
    .....
    protected HttpClient setupConnection(URL u, HttpMethodBase httpMethod, HTTPSampleResult res) {
        .....
        HostConfiguration hc = new HostConfiguration();
        hc.setHost(host, port, protocol);
        .....
        hc.setProxy(PROXY_HOST, PROXY_PORT);
        .....
        httpClient.getState().setProxyCredentials(
            newAuthScope(proxyHost, proxyPort, null, AuthScope.ANY_SCHEME),
            new NTCredentials(user, getProxyPass(), localHost, PROXY_DOMAIN)
        );
        .....
    }
}

```

Fig. 3. Two code snippets of “setProxy” returned by Krugle code search engine.

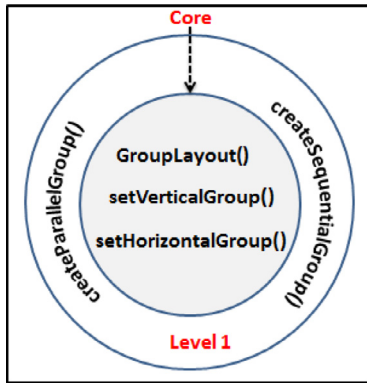


Fig. 4. An Example of multi-level frequent usage pattern consisting of core and peripheral (here Level 1) identified from SWING API.

of a large number of methods. For example in Swing API, the users should focus on 5 methods (3 core methods and 2 peripheral methods) to build Swing GUI interface instead of 30 methods. Then the users, as needed, change the properties of that interface using other methods of *GroupLayout* class.

3. The proposed approach (IML-FUP)

This section describes how the IML-FUP approach identifies multi-level frequent usage patterns of API of interest using formal concept analysis.

3.1. Approach overview

As shown in Fig. 5, the IML-FUP approach takes as input source code of both client programs and API of interest. The output is a set of multi-level frequent usage patterns embedded in that API.

The presented approach follows four steps as shown in Fig. 5. In the first step, the source code of API and client programs are statically analyzed to find usage transactions between the methods of client programs and the public methods of API. Such analyzing is performed using Eclipse Java Development Tool (JDT). In the sec-

ond step, co-usage API's methods are grouped together by building GSH-lattice. In the third step, this lattice is parsed to extract API's methods that constitute core and peripheral parts of each pattern through extracting core and peripheral concepts. Finally, the peripheral methods are organized into usage cohesion levels to form a multi-level frequent usage pattern.

3.2. Identifying usage transactions between client methods and API methods

An usage transaction is an interaction between a client program and an API of interest. This interaction is performed using method calling initiated by a method in client program source code for calling API's method(s). To identify the usage transactions, source code of client programs is statically analyzed. This static analysis allows to finding references between methods of client programs and public methods of a given API. The static analysis is performed using the Eclipse Java Development Tool (JDT).

This step takes as input source code of client programs and API of interest. Then, all client methods and their associated API's methods (i.e., caller and callee methods respectively) are identified using JDT according to Algorithm 1. This algorithm works by parsing each method in each client program source code.

3.3. Grouping Co-Usage API's methods by FCA

FCA is used to analyze usage transactions between clients' methods and API's methods, hierarchically organize these transactions as a GSH-concept lattice in order to group together Co-Usage API's methods that constitute a usage pattern. To end this, the formal context is defined as follows. Clients' methods represent the *objects* in the formal context. API's methods represent *attributes*. The relation between a client method and an API's method represents a usage transaction between those methods. Table 3 is an example of a formal context consisting of 9 clients' methods and 8 API's methods. The cross sign denotes to the existence of a usage transaction between those methods. Such formal context summarizes usage transactions between clients' methods and their associated API's methods.

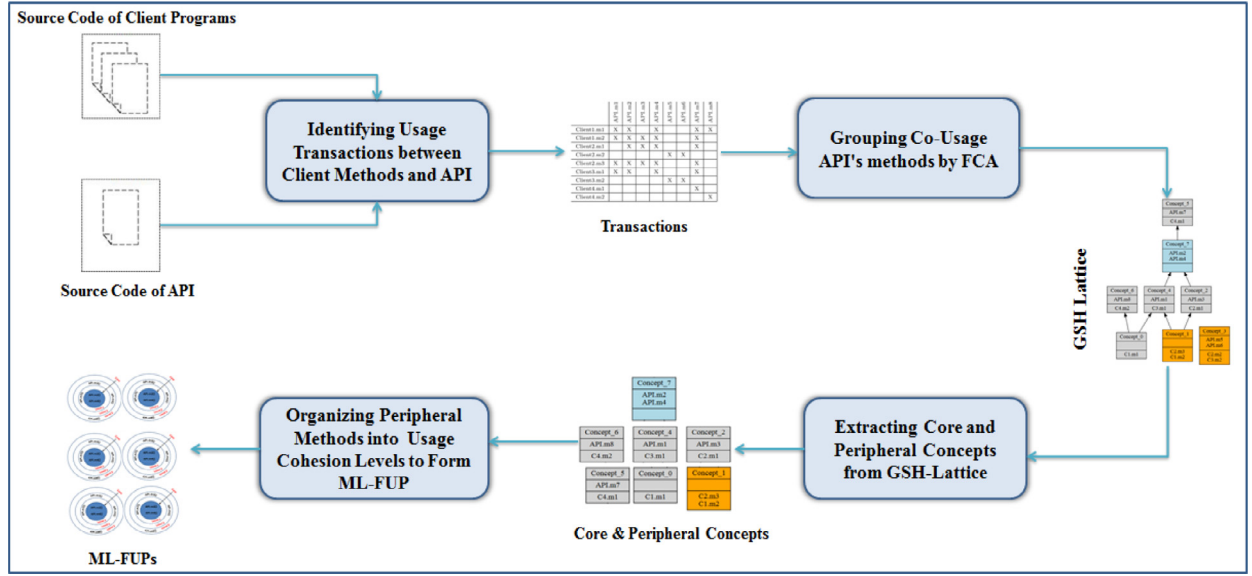


Fig. 5. The proposed approach overview.

Algorithm 1: Usage Transactions Identification.

Input: Source Code of All Client Programs (clients), Source Code API of Interest (api)

Output: allUsageTransactions

```

1 foreach clientProgram  $cp \in clients$  do
2   Set clientTransactions  $\leftarrow \phi$ 
3   foreach class  $c \in cp$  do
4     foreach method  $m \in c$  do
5       if  $m$  call  $api$  then
6         clientTransactions.add( $m$ ,
7           getCalledMethods( $api$ ))
8       end
9     end
10  end
11  allUsageTransactions.add(clientTransactions)
12 end
13 return allUsageTransactions

```

According to the above definition of the formal context, the generated GSH-lattice contains concepts that are composed of a set of clients' methods sharing a set of usage transactions of API's methods. This lattice represents a hierarchical organization of clients' method and their associated API's methods, so that a certain concept inherits its extent (clients' methods) from its descendants (sub-concepts) and its intent (API's methods) from its ascendants (super-concepts). Such a GSH-lattice represents usage

transactions between clients' method and API's methods. Fig. 6 shows the corresponding GSH-lattice of the formal context shown in Table 3. Based on this GSH-lattice and FCA definitions (see Section 2.1), the following facts are observed:

- Each concept having two or more API's methods in its intent, is called *core concept*. The intent of such concept represents a set of API's methods that are always called together and thus constitute the core part of a frequent usage pattern. For example, Concept_3 and Concept_7 in Fig. 6 are core concepts.
- Each an isolated core concept (having no sub-concepts and super-concepts), represents a frequent usage pattern without peripheral part. For example Concept_3 in Fig. 6.
- Each concept having only one API's method and downwardly reachable from a given core concept, is called *peripheral concept*. This is because the intents (API's methods) of such concepts frequently are invoked with the core concept methods. Therefore, these intents are members in the peripheral part. For example, Concept_4 and Concept_2 in Fig. 6 are peripheral concepts for the core concept Concept_7 because they are downwardly reachable from the Concept_7. All concepts downwardly reachable (either having intents or not) from a given core concept are called *downwardly reachable set* for referencing to it later in the text.
- Each concept having only one API's method and upwardly reachable from any concept in downwardly reachable set, is called *peripheral concept*, too. This is because the intents (API's methods) of such concepts are frequently invoked with the core concept methods. Therefore, these intents are members in the

Table 3

A formal context describing usage transactions between clients' methods and API's methods.

	API.m1	API.m2	API.m3	API.m4	API.m5	API.m6	API.m7	API.m8
Client1.m1	X	X		X			X	X
Client1.m2	X	X	X	X			X	
Client2.m1		X	X	X			X	
Client2.m2					X	X		
Client2.m3	X	X	X	X			X	
Client3.m1	X	X		X			X	
Client3.m2					X	X		
Client4.m1							X	
Client4.m2								X

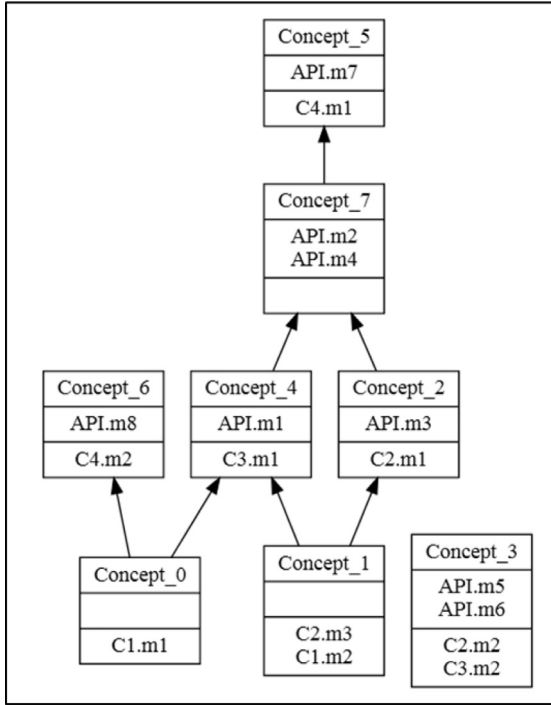


Fig. 6. The GSH-Lattice for the formal context defined in Table 3.

peripheral part. For example, the downwardly reachable set of core concept *Concept_7* is {*Concept_0*, *Concept_2*, *Concept_4*, *Concept_1*} and the *Concept_6* is upwardly reachable from the *Concept_0*. Therefore the intent of *Concept_6* (*API.m8*) is a member of the peripheral part.

- Each concept having only one API's method and upwardly reachable from a given core concept, is called *peripheral concept*, too. This is because the intents (API's methods) of such concepts frequently are invoked with the core concept methods. Therefore, these intents are members in the peripheral part. For example, *Concept_5* in Fig. 6 is peripheral concept for the core concept *Concept_7* because it is upwardly reachable from the *Concept_7*.
- Although API's methods in the intents of all upwardly reachable concepts from a given core concept are always called together with the core part methods by all clients' methods of that core, these API's methods should be considered as members of the peripheral part. This is because they are called by other client's methods that do not call the methods of the core part. For example, the API method *API.m7* is the intent of the *concept_5* which is upwardly reachable from the core concept *Concept_7*. This method should be considered as a member of the peripheral part because it is called by a client's method *C4.m1* which does not call the core part methods.
- The extents of both the core concept and concepts in downwardly reachable set of that core concept, represent all client's methods of API's methods (intent) in that core concept (all clients of the identified pattern). For example, the extents (*C3.m1*, *C2.m1*, *C1.m1*, *C2.m3*, *C1.m2*) of *Concept_0*, *Concept_1*, *Concept_2* and *Concept_4* are client's methods of API's methods (*API.m2* and *API.m4*) in intent of the core concept *Concept_7*.

3.4. Extracting core and peripheral concepts from GSH-lattice

The GSH-lattice of clients' methods and API's methods generated in the previous step is used to extract frequent usage patterns following steps below:

1. **Step I:** Determine a set of GSH-lattice concepts which represent the core concepts. The intent of each core concept represents the core part methods of a usage pattern.
2. **Step II:** For each core concept, determine a set of GSH-lattice concepts which represent the peripheral concepts corresponding to that core concept. All intents of peripheral concepts represent peripheral part methods of a usage pattern.

To realize the Step I, Algorithm 2 is proposed. This algorithm

Algorithm 2: Determine Core Concepts.

Input: *lattice*
Output: *coreConcepts*

```

1 List allConcepts ← getConceptList(lattice) foreach
  Concept ∈ allConcepts do
2   if (Concept.intentSize() > 1) then
3     coreConcepts.add(Concept)
4 return coreConcepts

```

takes as input GSH-lattice. The algorithm returns as output a list of core concepts. The function *getConceptList(lattice)* parses the input lattice and returns all lattice concepts. Then, each concept is checked to find the core concepts. For example, by applying this algorithm to the GSH-lattice shown in Fig. 6, the algorithms returns the following two core concepts: *Concept_3* and *Concept_7*.

Algorithm 3 is proposed to perform Step II. This algorithm takes

Algorithm 3: Determine Peripheral Concepts.

Input: *coreConcepts*, *lattice*
Output: FUPs // FUPs: Frequent Usage Patterns

```

1 FUPs ← ∅ // container combines each core with its
  corresponding peripheral concepts
2 foreach core ∈ coreConcepts do
3   if (core is non-isolated concept) then
4     Set PeripheralConcepts ← ∅
5     Set DRCs ← ∅ // DRCs: a set of Downward Reachable
      Concepts from core
6     DRCs ← DFS(core, lattice) //DFS: Depth First Search
7     DRCs.add(core)
8     PeripheralConcepts ← DRCs
9     foreach con ∈ DRCs do
10      Set URCs ← ∅ // URCs: is a set of Upward
        Reachable Concepts from concept
11      URCs ← DFS(con, lattice)
12      PeripheralConcepts ← URCs
13      PeripheralConcepts.remove(core)
14      FUPs.add(core, PeripheralConcepts)
15   else
16     FUPs.add(core, phi)
17 return FUPs

```

input a list of core concepts computed in Step I (*coreConcepts*) and GSH-lattice (called *lattice*). The algorithm is based on depth-first search (DFS) to determine as output all peripheral concepts for each non-isolated core concept. The algorithm works by finding for each core concept (called *core*) a set (called DRCs) of all downward reachable concepts from that core (Lines 2–7). Then, for each concept (called *con*) in DRCs the algorithm find a set (called URCs) of all upward reachable concepts from *con* (Lines 8–13). This collection of DRCs and their associated URCs' sets corresponding to each core concept contains all peripheral concepts of that core.

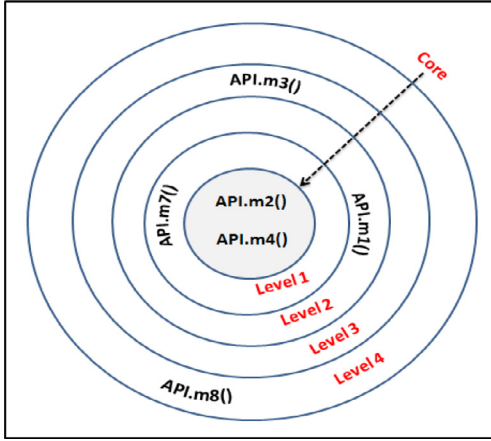


Fig. 7. Graphical representation of identified multi-level frequent usage pattern from GSH-lattice presented in Fig. 6.

The presented algorithm uses the function $DFS()$ two times to execute the depth first search algorithm. In the first time (see Line 6), the function computes DRCs set for each core concept. Therefore, each core concept is the root of new tree of DFS. The concepts that constitute each new tree represent DRCs of that core. In the second time (see Line 10), the function compute for each concept in DRCs its corresponding URCs set. Therefore, each concept in DRCs is the root of new tree of DFS. The concepts that constitute each new tree represent URCs of that concept. For example, the algorithm produces the following peripheral concepts for the core concept (*Concept_7*) shown in Fig. 6: {*Concept_5*, *Concept_2*, *Concept_4*, *Concept_6*, *Concept_1*, *Concept_0*}. The DRCs set of this core is {*Concept_2*, *Concept_4*, *Concept_1*, *Concept_0*}. There are many URCs sets in this lattice. For instance, the URCs of *Concept_0* is {*Concept_4*, *Concept_6*, *Concept_5*}.

3.5. Organizing peripheral methods into usage cohesion levels to form multi-Level frequent usage pattern

In the previous step, each core concept is associated with a set of its corresponding peripheral concepts. The intent of a core concept constitutes API's methods of a pattern's core. Also, the intents of the peripheral concepts represent API's methods that constitute peripheral part of a frequent usage pattern. In this step, these peripheral methods are organized into levels to form a multi-level frequent usage pattern. To end this, usage cohesion degree (UCD) with a pattern's core is computed for each API's method in peripheral part according to the following equation:

$$UDC(m) = \frac{|CM \cap CC|}{|CC|} \times 100\% \quad (1)$$

Based on UCD values, API's methods are organized into levels called usage cohesion levels. These levels are arranged from the highest UCD (the nearest to pattern's core) to the lowest UCD (far from the pattern's core) to produce a multi-level frequent usage pattern.

For example, Fig. 7 shows one multi-level frequent usage pattern identified from GSH-lattice presented in Fig. 6. This pattern consists of core and four levels of usage cohesion. The core contains *API.m2* and *API.m4* since they are always co-used together. The first level in the peripheral part consists of only one method *API.m7*. The second level consists of only one method *API.m1*. The third level consists of only one method *API.m3*. Finally, the fourth level consists of only one *API.m8*. This multi-level frequent usage pattern can be interpreted as follows. These four levels reflect four interference scenarios: (1) the core methods always are co-used

with *API.m7*, which shares 5/5 common client methods (*C1.m1*, *C1.m2*, *C2.m3*, *C2.m1*, *C3.m1*); (2) the core methods usually can be co-used with *API.m1*, which shares 4/5 common client methods (*C1.m1*, *C1.m2*, *C2.m3*, *C3.m1*). (3) the core methods frequently can be co-used with *API.m3*, which shares 3/5 common client methods (*C1.m2*, *C2.m3*, *C2.m1*). (4) the core methods occasionally can be co-used with *API.m8*, which shares 1/5 common client methods (*C1.m1*).

4. Evaluation setting

This section describes the setting of experiments. Particularly, the research questions, the client programs and APIs, and the used metrics for evaluating the effectiveness of the IML-FUP approach are defined in this section.

4.1. Investigation questions and evaluation procedure

The goal of this work is to evaluate whether the IML-FUP approach can identify multi-level frequent usage patterns which are enough cohesive to help in learning and using APIs, and which are generalizable and used independently of the API contexts. Thus, the main research questions of this work as a follows:

- **RQ1:** to which extent API's methods of an identified pattern are invoked together by all clients of this pattern? This is known as pattern usage cohesion.
- **RQ2:** to which extent the identified patterns could be consistent in contexts of other new client programs that are not considered in the identification process? This is known as pattern usage consistency.
- **RQ3:** to which extent the proposed approach is effective when it is compared to the existing ones?

In order to address the first research question (**RQ1**), a measure for a pattern usage cohesion is needed. For this, Service Interface Usage Cohesion metric (SIUC) is used (Pereplechikov et al., 2010). In this metric a service is deemed to be externally cohesive when all of its service operations are invoked by all the clients of this service. This definition for service cohesion is similar to pattern usage cohesion so SIUC metric is referred to as Multi-Level Pattern Usage Cohesion (MLPUC) in this work. MLPUC's values take a range in [0..1]. The larger the value of MLPUC is, the better the usage cohesion. The ideal usage cohesion occurs when MLPUC is equal to 1. This means that all the pattern's methods are actually always used together. For a given pattern p , MLPUC is defined as follows:

$$MLPUC(p) = \frac{\sum_{client} \frac{|PMe(client)|}{|PMe|}}{|clients(p)|} \quad (2)$$

where $|clients(p)|$ is the total number of all client methods of the API's methods in p . $|PMe|$ is the number of all API's methods in p . $|PMe(client)|$ is the number of API's methods in p invoked by a client method $client$.

In order to address the second research question (**RQ2**), a measure for a pattern usage consistency is needed to know whether the identified pattern from training client programs will have exact or similar usage cohesion in the context of new client programs (validation client programs) of API. For this, the following heuristic is considered: "identified patterns are considered consistent if their usage cohesion values in training client programs are equal or similar to their usage cohesion values in validation client programs". This is regardless of the contexts and features of those new client programs. Based on this heuristic, leave-one-out cross-validations for all the selected APIs are performed. Let N is the number of used client programs for the considered API, N runs on the API is conducted. Each run uses $N-1$ client programs as training client programs for identifying usage patterns, and leaves away one of the

API's client programs as a validation client program. Then, a pattern usage consistency is evaluated in two steps:

- In the first step, the cohesion of the identified usage patterns in the contexts of training and validation client programs is computed (based on MLPUC metric). However, for a given run, it is expected that some usage patterns identified from training client programs consists of only methods that are not used at all by the validation client programs. Therefore, to evaluate the consistency of usage patterns identified from training client programs in a run, only patterns which contain at least one method that is actually used by the validation client program are taking into account.
- In the second step, for each run, the consistency between usage patterns identified from training client programs and their corresponding usage patterns identified from the validation client programs is computed. For a given pattern p , if the MLPUC value of p in the context of training client programs remain the same (or very similar) in the context of the validation client programs, this pattern is considered consistent. The consistency is computed as follows (Saied et al., 2015b):

$$\text{Consistency}(p) = 1 - |\text{MLPUC}_T(p) - \text{MLPUC}_V(p)| \quad (3)$$

Where $\text{MLPUC}_T(p)$ and $\text{MLPUC}_V(p)$ are the usage cohesion values of the pattern p in the training client programs context and validation client programs context, respectively. The values of this metric take a range in $[0..1]$. When its value equals to 1 this means that the pattern p is *consistent*. When its value equals to 0 this indicates that the pattern p is *not consistent* and can not be generalizable in the context of new client programs.

In order to address the third question (RQ3), the proposed approach (IML-FUP) is compared to the most recent and relevant approach in the subject called *MLUP* which is presented in Saied et al. (2015b). *MLUP* extracts APIs' method calls from client programs using the static analysis. Then, it employs a clustering algorithm called *hierarchical DBSCAN* to detect groups of APIs' methods that are cohesive and consistent in terms of usage by client programs. These groups represent frequent usage patterns organized hierarchically. For an effective comparison, the APIs and client programs which were used to evaluate *MLUP* are also used to evaluate the effectiveness of *IML-FUP* in addition to other client programs with the same experiments set up in Saied et al. (2015b). Such a comparison allows to better position *IML-FUP*.

The obtained results of the *IML-FUP* approach mainly are evaluated in terms of the following metrics: *pattern usage cohesive* (*MLPUC*) and *pattern usage consistency* (*Consistency*). The above-mentioned metrics help to answer research questions addressed in this work. The results are not computed for the core and the peripheral patterns separately, but rather for the usage pattern as a whole.

4.2. Data collection

A collection of 89 Java projects from open-source repositories¹² are used as client programs to the identification process. These projects are different in terms of size and domains, such as *modeling applications*, *news applications*, *web applications*, *editors* and so on. Tables 4–7 give descriptive information of these projects. They are developed using different Java APIs. For experimental evaluation, four widely used APIs are considered: *httpclient*, for supporting communication with web services; *security*, for supporting security framework; *swing*, for providing a graphical user interface

Table 4
Client programs of SWING API.

API	Category	Client program names	Sum
swing	Modeling	SweetHome3D-3.4, Mogwai, Art-of-Illusion, Metawidget, Code2uml	5
	Editor	LaTeXDraw2.0.8, Freemind, GLIPS, JEdit, JHotDraw	5
	Web App.	Paros, RESTEasy, xsmile, Java chat	4
	Project Management	Msproject, Pert, GanttProject Core	3
	E-Mail	Davmail, Mailcarbon	2
	Tool	RapidMiner, Apache-jmeter 2.11, EasyFileShare, Open-so-frontend, PlotDigitizer, Pmd	6
	Library	G4P, Swingx,	2
	Network Framework	Neuroph	1
	Simulation	VASSAL-3.2.15	1
	Maps	AtlasCreator	1

Table 5
Client programs of HTTPCLIENT API.

API	Category	Client program names	Sum
httpclient	Web App.	Heritrix, Paros, OpenLaszlo	3
	E-Mail	Davmail	1
	Library	JavaBook-client, Weibo4j-oauth2	2
	Tool	Rabbitmq-management, Sage-gateway	2
	Network Framework	Mule-3.x	1
	News App.	RSSOwl	1
	Web Security	HueMorseCommunicator, HttpClientAuthHelper	2

Table 6
Client programs of AWT API.

API	Category	Client program names	Sum
awt	Web App.	Paros, RESTEasy, xsmile, Java chat, OpenLaszlo	5
	Modeling	SweetHome3D-3.4, Mogwai, Art-of-Illusion, Metawidget	4
	Editor	LaTeXDraw2.0.8, Freemind, GLIPS, JEdit, JHotDraw	5
	Project Management	Msproject, Pert, GanttProject Core, Htmlpdf	4
	E-Mail	Davmail, Mailcarbon	2
	Library	G4P, Swingx,	2
	Tool	RapidMiner, Apache-jmeter 2.11, EasyFileShare, Open-so-frontend, Pmd	5
	Network Framework	Neuroph	1
	Simulation	VASSAL-3.2.15	1
	Maps	AtlasCreator	1

Table 7
Client programs of SECURITY API.

API	Category	Client program names	Sum
security	Web App.	Heritrix, Paros, RESTEasy, Wildfly, xsmile, OpenLaszlo	6
	E-Mail	Davmail	1
	Library	YaHPConverter, ApacheJackrabbit,	2
	Tool	Apache-jmeter 2.11, Hibernate, Xstream	3
	Network Framework	MinaSource, Mule-3.x	2
	News App.	RSSOwl	1
	Web Security	Lcrypto, HttpClientAuthHelper	2

¹² <https://sourceforge.net/>, <https://github.com/>, <https://code.google.com/archive/search> and <https://gitorious.org/>

Table 8
Source code statistics of APIs of interest.

API name	#classes & interfaces	#Public methods
swing	2570	18,515
httpclient	871	7290
security	688	5764
awt	491	4778

(GUI); and *awt*, for creating user interfaces and for painting graphics and images. Table 8 gives descriptive statistics of these APIs.

To perform the study, 30 projects are selected for the *swing* and *awt* APIs, 12 projects for the *httpclient* API and 17 projects for *security* API as client programs (see Tables 4–7).

5. Results analysis

In this section, the obtained results of the IML-FUP approach are analyzed and the research questions formulated in Section 4.1 are answered.

Before studying in details the obtained results, it is better to provide statistical information about the identified usage patterns of each API. Table 9 shows, for each API, number of identified patterns and size of these patterns in terms of minimum, average and maximum of API's methods at different levels of usage cohesion (i.e., patterns with only core part, patterns with only one level in addition to core part and patterns with only two levels in addition to core part, respectively). It is noticed that number of identified patterns from both *swing* and *awt* (100 and 60, respectively) is more than patterns identified from both *security* and *httpclients* (37 and 36, respectively). This is because the number of clients programs of *swing* and *awt* (30) is more than clients of *security* and *httpclients* (17 and 13, respectively). Also, it is important to note that when number of levels of identified patterns increases, the size of these patterns in terms of average and maximum also increases by adding new API's methods in that levels.

5.1. Usage cohesion of patterns identified (RQ1)

To answer the first research question RQ1, the co-usage relationships between API's methods of each identified pattern should be analyzed and evaluated. To end this, the MLPUC metric defined in Section 4.1 is used to compute usage cohesion values for all identified usage patterns per studied API. Table 10 shows for each studied API average usage cohesion values of identified patterns at different sizes (*core*, *Level 1* and *Level 2*). The usage cohesion values indicate to strong usage cohesion of identified patterns for each API. Indeed, these values are 100% for patterns consisting of only core, between 96% and 100% for patterns consisting of single level in addition to core, and between 92% and 96% for patterns consisting of two levels in addition to core.

It is clear from Table 10 when the size of identified patterns increases, the average usage cohesion of these patterns decreases. This is because adding more levels to the pattern core means adding new API's methods having different usage cohesion values with core's methods. These values decrease when number of levels increases as these levels represent usage cohesion levels. For example, usage cohesion values of API's methods belonging to Level 2 are less than usage cohesion values of API's methods belonging to Level 1.

Fig. 8 represents a graphical representation of average usage cohesion values of different sizes of patterns identified from each API against the numbers of clients programs used by that API. It is important to notice that these values are equal to 1 for all APIs' patterns consisting only from core and similar for multi-level patterns although the number of clients of these are significantly different

(*httpclint* uses 12 programs while *swing* uses 30 programs). This means that the IML-FUP approach can identify strong usage cohesion patterns from a given API even if there are only a handful of client programs of that API.

In summary, the answer to the research question RQ1 is that the IML-FUP approach identifies multi-level usage patterns that have strong co-usage relationships between API's methods of each identified pattern at different sizes. The answer is based on the result of average usage cohesion values of these patterns shown in Table 10.

5.2. Consistency of patterns identified (RQ2)

To answer the second research question (RQ2), leave-one-out cross-validations for all the selected APIs should be conducted to observe the usage cohesion for patterns identified from validation clients programs. Then, usage cohesion values for patterns identified from both validation and training are compared by using the consistency metric defined in Section 4.1 in order to characterize the consistency of these patterns.

5.2.1. Patterns usage cohesion in validation clients

Table 11 shows average usage cohesion values of identified patterns in the context of validation clients programs. It is notice that these average values are high regardless of the size of identified patterns. The average values for patterns' core are equal to 1 while the average values for patterns consisting of one and two levels in addition to core take a range in (95%–99%) and (91%–98%), respectively. Also, It can be seen in Table 11 that the standard deviation values are very low and maximum values of usage cohesion are very high (equal to 100%) regardless of the size of identified patterns. This indicates that the IML-FUP approach identifies usage patterns that always have strong cohesion in the context of potential validation programs.

Based on the result shown in Table 11, API usage patterns identified by the IML-FUP approach retains their informative criteria across different new client programs. Thus, these patterns can be used to enrich the API documentation with co-usage relationships with high confidence without a need to take into account all possible usage contexts (client programs) of the API of interest.

5.2.2. Patterns consistency

Table 12 shows average consistency values of identified patterns at different sizes for all studied APIs. These values indicate that the identified patterns are characterized with high consistency. Indeed, these average values across multiple validation client programs takes a range in (94%–100%) for patterns'core, (84%–92%) for single level patterns and (81%–90%) for two levels patterns. The table also shows that maximum consistency values for most of studied APIs at different pattern sizes equal to the ideal value, which is 100%, and the standard deviation values are very small. The case of *swing* API is considered as an example, where 30 leave-one-out cross validations have been made. In this case, the results show that, for any *swing*'s identified pattern, on average 91.3% of co-usage relationships between pattern's methods remain the same across 30 validation client programs of *swing*.

Fig. 9 represents a graphical representation of average consistency values of different sizes of patterns identified from each API against the numbers of clients programs used by that API. It is important to notice that these values are equal to 1 for all APIs' patterns consisting only from core and similar for multi-level patterns although the number of clients of these APIs are significantly different (*httpclint* uses 12 programs while *swing* use 30 programs). This means that the IML-FUP approach can identify high consistent patterns from a given API even if there are only a handful of client programs of that API.

Table 9
Statistical information of the identified patterns per API.

API	Core				Level 1				Level 2			
	UP's size				UP's size				UP's size			
	UPs	Avg	Min	Max	UPs	Avg	Min	Max	UPs	Avg	Min	Max
security	37	3.86	2	25	37	6.35	2	34	37	7.08	2	34
httpClient	36	2.83	2	9	36	6.86	2	19	36	8.03	2	19
swing	100	2.76	2	14	100	6.97	2	35	100	8.04	2	46
awt	60	2.61	2	6	60	6.2	2	20	60	6.8	2	20

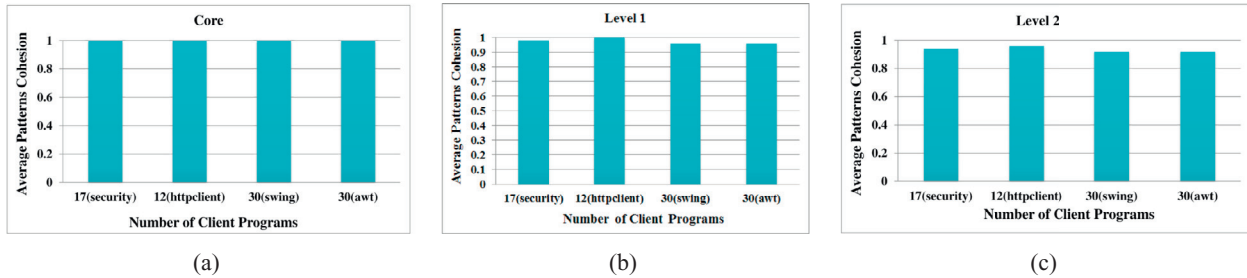


Fig. 8. Average usage cohesion of identified patterns at different number of client programs for each API.

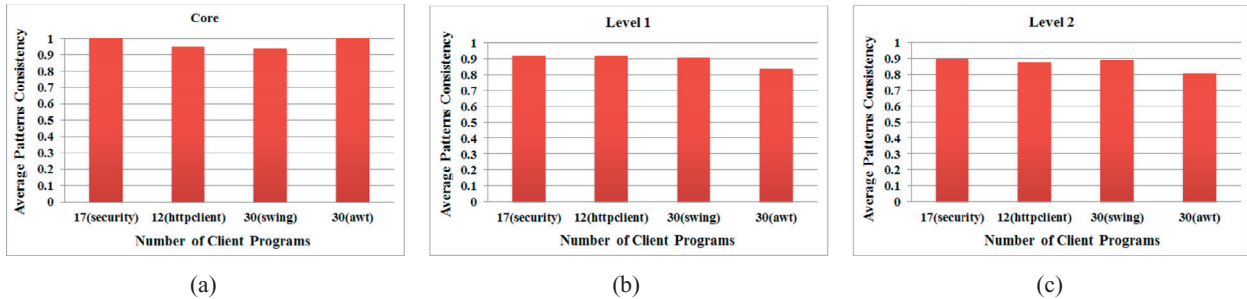


Fig. 9. Average consistency of identified patterns at different number of client programs for each API.

Table 10
Average cohesion of identified patterns per API.

API	Core	Level 1	Level 2
security	1.00	0.98	0.94
httpClient	1.00	1.00	0.96
swing	1.00	0.96	0.92
awt	1.00	0.96	0.92

5.3. The effectiveness of IML-FUP against the most recent and relevant work in the subject (RQ3)

To answer the third question (RQ3), usage cohesion and consistency for all identified usage patterns per API are summarized in [Tables 13](#) and [14](#) for compared approaches (*IML-FUP* and *MLUP*).

[Table 13](#) shows, for each studied API, average usage cohesion values of different sizes of identified patterns using two approaches. The averages values indicate that *IML-FUP* outperforms *MLUP* for identifying cohesive usage patterns. Indeed, these values for *IML-FUP* are between 96% and 100% for patterns consisting of the Level 1 (of course, in addition to the *core* level) while *MLUP*'s average values are between 94% and 96% for patterns consisting of the Level 1. This means that the worst case in *IML-FUP* is the best case in *MLUP*. Also, it is clear from [Table 13](#) that average cohesion values (92%–96%) of *IML-FUP*'s usage patterns having Level 2 (of course in addition to Level 1 and the *core* level) are very close to average cohesion values (90%–96%) of *MLUP*'s usage

In summary, the answer to the research question RQ2 is that the *IML-FUP* approach identify multi-level usage patterns that have high consistency (i.e., co-usage relationships between API's methods of each identified pattern across new client programs remain the same or very close to those in the context of training programs). The answer is based on the result of average consistency values of these patterns shown in [Table 12](#).

Table 11
Statistics on the cohesion of identified patterns per API, in validation clients.

API	Core			Level 1			Level 2		
	Avg	StdDev	Max	Avg	StdDev	Max	Avg	StdDev	Max
security	1.0	0.0	1.0	0.99	0.01	1.0	0.98	0.03	1.0
httpClient	1.0	0.0	1.0	0.98	0.02	1.0	0.98	0.03	1.0
swing	1.0	0.0	1.0	0.97	0.02	1.0	0.95	0.03	1.0
awt	1.0	0.0	1.0	0.95	0.04	1.0	0.91	0.07	1.0

Table 12
Statistics on the consistency of identified patterns per API.

API	Core			Level 1			Level 2		
	Avg	StdDev	Max	Avg	StdDev	Max	Avg	StdDev	Max
security	1.0	0.0	1.0	0.92	0.02	0.95	0.90	0.0	0.90
httpClient	0.95	0.08	1.0	0.92	0.09	1.0	0.88	0.10	1.0
swing	0.94	0.07	1.0	0.91	0.06	1.0	0.89	0.07	1.0
awt	1.0	0.0	1.0	0.84	0.13	1.0	0.81	0.10	1.0

Table 13
Average cohesion of identified APIs' patterns, for IML-FUP and MLUP.

API	IML-FUP		MLUP
	Level 1	Level 2	Level 1
security	0.98	0.94	0.90
httpClient	1.00	0.96	0.96
swing	0.96	0.92	0.94
awt	0.96	0.92	0.94

patterns having Level 1. Taking into account usage cohesion values of API's methods belonging to Level 2 are less than usage cohesion values of API's methods belonging to Level 1, IML-FUP approach identifies patterns that are very strong co-usage relationships than those identified by MLUP.

Table 14 shows statistics information for usage consistency of identified patterns at different sizes by using the compared approaches. The results show that IML-FUP outperforms MLUP in terms of two aspects related to pattern usage consistency. On the one hand, average usage consistency values of IML-FUP's patterns that consist of Level 1 for all APIs are greater than MLUP's patterns consisting of the same number of levels. Indeed, the consistency average values of IML-FUP's patterns take a range in (84% and 92%) while consistency average values of MLUP's patterns take a range in (77% and 86%). This indicates that the best consistency average value achieved by MLUP (86%) is very close to the worst consistency average value achieved by IML-FUP (84%). The table also shows that maximum consistency values of IML-FUP's patterns are greater than maximum consistency values of MLUP's patterns with similar standard deviation values. On the other hand, the results show that IML-FUP can identify usage patterns having higher usage consistency and size than those identified by MLUP taking into account that the patterns with small number of levels have higher consistency than the patterns with large number of levels. As it is clear from Table 14 that average and maximum consistency values of IML-FUP's patterns consisting of Level 2 are higher than MLUP's pattern which consists of only Level 1 (Tables 15–18).

In summary, the answer to the third research question is that IML-FUP approach outperforms the most recent and relevant work in the subject (MLUP) in terms of usage cohesion and usage consistency for patterns identified from four widely used APIs. The answer is based on the result of average cohesion and consistency

values of these patterns shown in Tables 13 and 14. Also, drawbacks of MLUP are listed in details in the related work Section 6.

5.4. Threats to validity

The IML-FUP approach is subjected to two types of threats: internal validity and external validity. These threats are as follows:

5.4.1. Threats to internal validity

The internal threats of validity concern the IML-FUP approach are as follows:

1. The IML-FUP approach takes as input multiple client programs that are manually collected from different open source repositories. This may need a considerable amount of time. However, the experimental results show that the IML-FUP approach can work successfully in case of having a few clients programs of a given API. Also, there is no automated tool to collect client programs of a given API.
2. The IML-FUP approach uses static analysis technique to find usage transactions (API's method calls) between client programs and their APIs. However, this affects the results as it does not address polymorphism and dynamic binding. In fact, the most important dependencies between object-oriented client programs and their respective APIs are realized through methods calls. Thus, by not taking into account polymorphism and dynamic binding does not have a high impact on the overall results. In contrast, dynamic analysis addresses this limitation but the challenge with dynamic analysis is to identify all use cases of a given client programs.
3. As the IML-FUP approach is use-driven identification, the results (namely, the number of identified usage patterns from a given API) depend on the coverage of the API methods which are used by client programs. This coverage may be redundant (i.e., an API method may be used (covered) by numerous client methods from different client programs). However, this limitation is shared by all existing work that relies on client programs during the identification process (Saied et al., 2015b; Zhong et al., 2009).
4. The IML-FUP approach relies on client programs of an API of interest during the identification process. However, the source code of client programs is not always available or the client programs themselves do not exist yet for newly released APIs. This limitation is also shared by all existing work that relies on client programs during the identification process. Moreover,

Table 14
Statistics on the consistency of identified APIs' patterns, for IML-FUP and MLUP.

API	IML-FUP						MLUP		
	Level 1			Level 2			Level 1		
	Avg	StdDev	Max	Avg	StdDev	Max	Avg	StdDev	Max
security	0.92	0.02	0.95	0.90	0.0	0.90	0.86	0.09	0.98
httpClient	0.92	0.09	1.0	0.88	0.10	1.0	0.77	0.09	0.89
swing	0.91	0.06	1.0	0.89	0.07	1.0	0.83	0.05	0.92
awt	0.84	0.13	1.0	0.81	0.10	1.0	0.84	0.04	0.94

the non-client-based approaches (Saied et al., 2015a) are not an alternative solution to client-based ones. They are used when client programs are not available (i.e., for newly released APIs and not widely used APIs).

5.4.2. Threats to external validity

The external threats of validity concern the IML-FUP approach are as follows:

1. The IML-FUP approach is validated using only Java code (Java clients programs and APIs). However, it can be generally applicable for other object-oriented languages (e.g., c++, c#). The reason behind this generalization is that the approach relies on common object-oriented structures and relationships such as method calling.
2. The IML-FUP approach is applied on a limited set of APIs and client programs. Although the used APIs and their client programs are fair enough to validate the proposed approach, a larger set of case studies are necessary to better test the approach and generalize the result.

6. Related work

In the recent years, much research effort was directed to different aspects of APIs usage. Recently, Robillard et al. surveyed a wide range of techniques to automate the inference of API properties and knowledge (Robillard et al., 2013). The proposed techniques can be organized into the following categories according to the goal they pursue (Robillard et al., 2013; Saied et al., 2015b): recommendation of API elements, recommendation of API usage example, detecting bugs, detecting API usage obstacles, visualization of API usages and mining API usage patterns.

6.1. Recommendation of API elements

Recommendation of API elements, also known as source code completion, is one of the most popular integrated development environment (IDE) features for accessing APIs and freeing programmer from remembering every detail. Many approaches were proposed for enhancing current completion systems to work more effectively with large APIs (Bruch et al., 2009; Hou and Pletcher, 2011; Nguyen et al., 2012). Bruch et al. (2009) present an Intelligent Code Completion System (ICCS) that learns from existing code repositories. Their work aims at filtering the methods proposed by an auto-completion system based on the relevance of the current context and assessing the relevance of every proposed method. Hou and Pletcher (2011) implement a technique called Better Code Completion (BCC) that uses a combination of sorting, filtering and grouping of APIs. Their technique can sort completion proposals based on the type-hierarchy or frequency count of method calls. It can filter non-API public methods. Nguyen et al. (2012) use a graph-based algorithm to develop a context sensitive code completion technique, called GrePacc. The technique performs code completion based on a collected database of API usage patterns. It extracts the context-sensitive features from the code under editing. The features are then used to search and rank the patterns that are best matched with the current code. Recently Asaduzzaman et al. (2014) propose a context sensitive code completion technique called (CSCC) like GrePacc. However, it uses new sources of information which is the context of a target method call to make recommendations.

6.2. Recommendation of API usage example

Many approaches are interested in providing source code examples automatically for recommendation of API usage. These ap-

proaches can be organized into two categories: *IDE-based recommendation systems* and *JavaDoc-based recommendation systems*.

IDE-based recommendation systems, such as Duala-Ekoko and Robillard (2011) and Holmes et al. (2006), are implemented as IDEs' plug-ins. The main benefit of these systems is ability to explore the syntactic context provided by the IDE to recommend examples more relevant to developers. However, the examples proposed by these systems can not be used for documentation purpose, since they are dependent on a particular development context.

JavaDoc-based recommendation systems, such as Wang et al. (2011), Montandon et al. (2013) and Buse and Weimer (2012), are implemented independently from any IDE or platform and can be accessed from the web. These systems work by instrumenting API documentation with usage examples. They depend on a static slicing, clustering or pattern abstraction. As the key benefits, these systems are characterized as a wider reachability (because they can be accessed from the web) and greater scalability. The limitation of these systems is that they do not provide the same level of precision as IDE-based recommendation systems.

6.3. Detecting bugs

Identifying usage patterns can also be used to detect bugs. For example, if a proposed technique determines that API methods (*open()* and *close()*) should be invoked together within the same function, then the presence of an unmatched *close()* method is an indicator of a potential bug. Many techniques are proposed for detecting APIs bugs (Li and Zhou, 2005; Livshits and Zimmermann, 2005; Monperrus et al., 2010). Li and Zhou (2005) implemented a tool called PR-Miner to automatically detect usage patterns from large source code. These patterns are considered rules and used to find violations. The assumption is that rule violations can uncover bugs. Livshits and Zimmermann (2005) propose a tool called DynaMine to identify error patterns by mining the change history of an API's clients. This tool works by inferring pairwise association rules for methods inserted in a single source file revision. These rules are then checked by instrumenting and executing the client's source code. In Monperrus et al. (2010), Monperrus et al. implement a tool called Detector of Missing Method Calls (DMMC) to detect bugs through statistics information about type-usages. A type-usage is a list of method calls on a variable of a given type occurring somewhere within the context of a particular method body. They use this information to detect other client methods that may need to call the missing method.

6.4. Detecting API usage obstacles

Recently, a body of research work is directed to explore API usage obstacles through analyzing programmers questions in newsgroup, forums and Q&A website, such as *stackoverflow.com* (Hou and Li, 2011; Wang and Godfrey, 2013). This enables API designers to understand the problems that programmers face while using their APIs, and to make corresponding improvements to alleviate these problems.

Hou and Li (2011) analyze 172 programmer discussions collected from the *Swing Forum*, identify a set of API learning obstacles and describe in detail these obstacles and their probable causes. Wang and Godfrey (2013) analyze iOS and Android developer questions on *stackoverflow.com* website to detect API usage obstacles. They used topic models to summarize possible scenarios from all the posts related to previously identified API classes. They discovered API classes that often cause usage obstacles but are not frequently used. They also discovered a few scenarios that appear to be the common cause of API usage obstacles.

6.5. Visualization of API usages

From different perspective, many contributions were presented to enhance understanding API usage through visualizing relationships among API's elements, and between API's elements and their clients [Moritz et al. \(2013\)](#) and [Roover et al. \(2013\)](#). [Moritz et al. \(2013\)](#) implement a tool called *ExPort* to visualize relevant API usage examples. The idea behind *ExPort* is that whenever the programmer selects the API's method to use, *ExPort* suggests to the programmer usage examples related to the task visually. This allows navigating the results and reason about their relationships. [Roover et al. \(2013\)](#) propose a multi-dimensional exploration approach of API usage. The explored dimensions are related to the hierarchical organization of projects and APIs, metrics of API usage and API domains.

6.6. Mining API usage patterns for documentation and understanding

The most related contributions to the one presented in this article are those interested in mining API usage patterns for the purpose of documenting and understanding APIs ([Zhong et al., 2008](#); [Nguyen et al., 2009](#); [Michail, 1999](#); [Zhong et al., 2009](#); [Michail, 2000](#); [Uddin et al., 2012](#); [Wang et al., 2013](#)). They adopted different categories of API usage patterns, different techniques for mining patterns and different ways to assess their correctness and usefulness. There are three distinguished categories of API usage patterns: (1) temporal, (2) sequential and (3) unordered usage patterns. These patterns were assessed through *consistency*, *cohesion*, *coverage* and *succinctness*.

The limitations of above-mentioned techniques are inability to identify common or consistent API usage pattern across different new API clients or contexts. Also, the inability to accommodate or capture interference between API usage scenarios. The IML-FUP overcomes these limitations through the following contributions. First, the identified usage patterns are strongly cohesive and generalizable for a variety of API client programs. Second, The hierarchical construction (core and peripheral parts) of identified usage patterns accommodate interferences between API usage scenarios and contexts.

Recently, [Saied et al. \(2015b\)](#) proposed a technique called MLUP to mine multi-level API usage patterns from source code of multiple clients programs. Their technique used a clustering algorithm called DBSCAN to mine unordered usage patterns. The main drawbacks of Saied et al.'s technique are as follows. First, their technique depends on an expert intervention during the mining process to adjust values of two important parameters for DBSCAN and hierarchical DBSCAN algorithms called *epsilon* and *epsilonStep*, respectively. Second, their technique mines false positive usage pattern core. This is because it considers any API's method having UDC's value equal to 1 as a member of a pattern core. In fact, this is not always true see [Sections 3.3](#) and [3.5](#). Finally, their technique can not explicitly mine usage patterns that have only core patterns (without peripheral part). This is because their technique assumes that each pattern should have peripheral part. The benefits of the IML-FUP over their technique are as follows. First, IML-FUP overcomes all limitations of MLUP. Second, IML-FUP identifies usage patterns that are strongly cohesive and highly consistent across different client programs.

7. Conclusions and future work

7.1. Conclusions

In this article, an approach, called IML-FUP, is proposed to help developers understand API usage and facilitate the development

tasks when they use new APIs. It identifies API's multi-level frequent usage patterns from open source repositories automatically. In particular, IML-FUP uses formal concept analysis technique to combine API's methods which always or frequently are used together in code snippets to identify API usage patterns. An experimental evaluation is conducted using four APIs and 89 client programs. For all studied APIs, the experimental results show that IML-FUP identifies usage patterns that are, always, highly consistent and strongly cohesive across a considerable variation of client programs.

7.2. Future work

There are many research works can be investigated in the near future. These include conducting a more precise analysis using dynamic analysis which could improve the results, experimenting with a large number of APIs and validating the proposed approach with human experts to better validate the approach.

Appendix

In this section, different examples of multi-level usage patterns identified from studied APIs (SWING, AWT, SECURITY and HTTP-CLIENT) are presented as follows:

Table 15

An example of a multi-level usage pattern identified from SWING API.

	Usage pattern
Core	javax.swing.DefaultListSelectionModel.isSelectedIndex() javax.swing.DefaultListSelectionModel.getMinSelectionIndex() javax.swing.DefaultListSelectionModel.getMaxSelectionIndex() javax.swing.DefaultListSelectionModel.isEmpty()
Level 1	javax.swing.tree.DefaultTreeSelectionModel.clearSelection() javax.swing.JTree.getPathForRow()
Level 2	Empty

Table 16

An example of a multi-level usage pattern identified from AWT API.

	Usage pattern
Core	java.awt.event.MouseEvent.getYOnScreen() java.awt.event.MouseEvent.getXOnScreen()
Level 1	java.awt.Rectangle.contains() java.awt.event.MouseEvent.getPoint() java.awt.Point.translate() java.awt.event.MouseEvent.getModifiersEx() java.awt.Component.getBounds()
Level 2	Empty

Table 17

An example of a multi-level usage pattern identified from SECURITY API.

	Usage pattern
Core	javax.security.auth.spi.LoginModule.abort() javax.security.auth.spi.LoginModule.commit() javax.security.auth.spi.LoginModule.initialize() javax.security.auth.spi.LoginModule.login()
Level 1	Empty
Level 2	Empty

Table 18

An example of a multi-level usage pattern identified from HTTPCLIENT API.

	Usage pattern
Core	org.apache.commons.httpclient.HostConfiguration.getProxyPort() org.apache.commons.httpclient.HostConfiguration.getLocalAddress() org.apache.commons.httpclient.HostConfiguration.getProxyHost()
Level 1	org.apache.commons.httpclient.HostConfiguration.getPort() org.apache.commons.httpclient.HostConfiguration.getHost() org.apache.commons.httpclient.HostConfiguration.getProtocol()
Level 2	org.apache.commons.httpclient.params.DefaultHttpParams.setDefaults() org.apache.commons.httpclient.HttpConnection.setHttpConnectionManager() org.apache.commons.httpclient.HttpConnection.getParams() org.apache.commons.httpclient.protocol.Protocol.resolvePort()

References

- Asaduzzaman, M., Roy, C.K., Schneider, K.A., Hou, D., 2014. Csc: Simple, efficient, context sensitive code completion. In: Proceedings of the 30th International Conference on Software Maintenance and Evolution. IEEE, pp. 71–80. doi:10.1109/ICSME.2014.29.
- Barbut, M., Monjardet, B., 1970. *Ordre et classification : alg bre et combinatoire*. Hachette, Paris.
- Berry, A., Gutierrez, A., Huchard, M., Napoli, A., Sigayret, A., 2014. Hermes: a simple and efficient algorithm for building the aoc-poset of a binary relation. *Ann. Math. Artif. Intell.* 72 (1–2), 45–71. doi:10.1007/s10472-014-9418-6.
- Bhatti, M.U., Anquetil, N., Huchard, M., Ducasse, S., 2012. A catalog of patterns for concept lattice interpretation in software reengineering. In: SEKE. Knowledge Systems Institute Graduate School, pp. 118–123.
- Bruch, M., Monperrus, M., Mezini, M., 2009. Learning from examples to improve code completion systems. In: Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering. ACM, New York, NY, USA, pp. 213–222. doi:10.1145/1595696.1595728.
- Buse, R.P.L., Weimer, W., 2012. Synthesizing api usage examples. In: Proceedings of the 34th International Conference on Software Engineering. IEEE Press, Piscataway, NJ, USA, pp. 782–792.
- Duala-Ekoko, E., Robillard, M.P., 2011. Using structure-based recommendations to facilitate discoverability in apis. In: Proceedings of the 25th European Conference on Object-oriented Programming. Springer-Verlag, Berlin, Heidelberg, pp. 79–104.
- Ganter, B., Wille, R., 1999. *Formal Concept Analysis, Mathematical Foundations*. Springer-Verlag.
- Godin, R., Mili, H., 1998. Building and maintaining analysis-level class hierarchies using galois lattices. In: Proc. ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA). ACM Press.
- Han, J., 2005. *Data Mining: Concepts and Techniques*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.
- Holmes, R., Walker, R.J., Murphy, G.C., 2006. Approximate structural context matching: an approach to recommend relevant examples. *IEEE Trans. Softw. Eng.* 32 (12), 952–970. doi:10.1109/TSE.2006.117.
- Hou, D., Li, L., 2011. Obstacles in using frameworks and apis: an exploratory study of programmers' newsgroup discussions. In: Program Comprehension (ICPC), 2011 IEEE 19th International Conference on, pp. 91–100. doi:10.1109/ICPC.2011.21.
- Hou, D., Pletcher, D.M., 2011. An evaluation of the strategies of sorting, filtering, and grouping api methods for code completion. In: Software Maintenance (ICSM), 2011 27th IEEE International Conference on, pp. 233–242. doi:10.1109/ICSM.2011.6080790.
- HttpClient authentication. <https://hc.apache.org/httpcomponents-client-ga/tutorial/html/authentication.html>, Accessed: 2016-06-25.
- Kuznetsov, S., Obiedkov, S., 2002. Comparing performance of algorithms for generating concept lattices. *J. Exp. Theor. Artif. Intell.* 14, 189–216.
- Li, Z., Zhou, Y., 2005. Pr-miner: automatically extracting implicit programming rules and detecting violations in large software code. *SIGSOFT Softw. Eng. Notes* 30 (5), 306–315. doi:10.1145/1095430.1081755.
- Livshits, B., Zimmermann, T., 2005. Dynamine: finding common error patterns by mining software revision histories. *SIGSOFT Softw. Eng. Notes* 30 (5), 296–305. doi:10.1145/1095430.1081754.
- Michail, A., 1999. Data mining library reuse patterns in user-selected applications. In: Automated Software Engineering, 1999. 14th IEEE International Conference on, pp. 24–33. doi:10.1109/ASE.1999.802089.
- Michail, A., 2000. Data mining library reuse patterns using generalized association rules. In: Software Engineering, 2000. Proceedings of the 2000 International Conference on, pp. 167–176. doi:10.1145/337180.337200.
- Monperrus, M., Bruch, M., Mezini, M., 2010. Detecting missing method calls in object-oriented software. In: Proceedings of the 24th European Conference on Object-oriented Programming. Springer-Verlag, Berlin, Heidelberg, pp. 2–25.
- Montandon, J.E., Borges, H., Felix, D., Valente, M.T., 2013. Documenting apis with examples: Lessons learned with the apiminer platform. In: 2013 20th Working Conference on Reverse Engineering (WCORE), pp. 401–408. doi:10.1109/WCORE.2013.6671315.
- Moritz, E., Linares-V asquez, M., Poshyvanyk, D., Grechanik, M., McMillan, C., Gethers, M., 2013. Export: Detecting and visualizing api usages in large source code repositories. In: Automated Software Engineering (ASE), 2013 IEEE/ACM 28th International Conference on, pp. 646–651. doi:10.1109/ASE.2013.6693127.
- Nguyen, A.T., Nguyen, T.T., Nguyen, H.A., Tamrawi, A., Nguyen, H.V., Al-Kofahi, J., Nguyen, T.N., 2012. Graph-based pattern-oriented, context-sensitive source code completion. In: Proceedings of the 34th International Conference on Software Engineering. IEEE Press, Piscataway, NJ, USA, pp. 69–79.
- Nguyen, T.T., Nguyen, H.A., Pham, N.H., Al-Kofahi, J.M., Nguyen, T.N., 2009. Graph-based mining of multiple object usage patterns. In: Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on The Foundations of Software Engineering. ACM, New York, NY, USA, pp. 383–392. doi:10.1145/1595696.1595767.
- Perepletchikov, M., Ryan, C., Tari, Z., 2010. The impact of service cohesion on the analyzability of service-oriented software. *IEEE Trans. Serv. Comput.* 3 (2), 89–103.
- Poshyvanyk, D., Marcus, A., 2007. Combining Formal Concept Analysis with Information Retrieval for Concept Location in Source Code. Washington, DC, USA, pp. 37–48.
- Robillard, M.P., Bodden, E., Kawrykow, D., Mezini, M., Ratchford, T., 2013. Automated api property inference techniques. *IEEE Trans. Softw. Eng.* 39 (5), 613–637. doi:10.1109/TSE.2012.63.
- Roover, C.D., L  mmel, R., Pek, E., 2013. Multi-dimensional exploration of api usage. In: 2013 21st International Conference on Program Comprehension (ICPC), pp. 152–161. doi:10.1109/ICPC.2013.6613843.
- Saied, M.A., Abdeen, H., Benomar, O., Sahraoui, H., 2015. Could we infer unordered api usage patterns only using the library source code? In: Proceedings of the 2015 IEEE 23rd International Conference on Program Comprehension. IEEE Press, Piscataway, NJ, USA, pp. 71–81.
- Saied, M.A., Benomar, O., Abdeen, H., Sahraoui, H., 2015. Mining multi-level api usage patterns. In: 2015 IEEE 22nd International Conference on Software Analysis, Evolution, and Reengineering (SANER), pp. 23–32. doi:10.1109/SANER.2015.7081812.
- Shatnawi, A., Seriai, A.-D., Sahraoui, H., Alshara, Z., 2016. Reverse engineering reusable software components from object-oriented {APIs}. *J. Syst. Softw.* <http://dx.doi.org/10.1016/j.jss.2016.06.101>.
- The jakarta commons httpclient component. <http://hc.apache.org/httpclient-3.x/> Accessed: 2016-06-25.
- Uddin, G., Dagenais, B., Robillard, M.P., 2012. Temporal analysis of api usage concepts. In: 2012 34th International Conference on Software Engineering (ICSE), pp. 804–814. doi:10.1109/ICSE.2012.6227138.
- Valtchev, P., Missaoui, R., Godin, R., 2004. *Formal Concept Analysis for Knowledge Discovery and Data Mining: The New Challenges*. In: Lecture Notes in Computer Science, 2961. Springer, pp. 352–371.
- Wang, J., Dang, Y., Zhang, H., Chen, K., Xie, T., Zhang, D., 2013. Mining succinct and high-coverage api usage patterns from source code. In: Mining Software Repositories (MSR), 2013 10th IEEE Working Conference on, pp. 319–328. doi:10.1109/MSR.2013.6624045.
- Wang, L., Fang, L., Wang, L., Li, G., Xie, B., Yang, F., 2011. Apiexample: an effective web search based usage example recommendation system for java apis. In: Automated Software Engineering (ASE), 2011 26th IEEE/ACM International Conference on, pp. 592–595. doi:10.1109/ASE.2011.6100133.
- Wang, W., Godfrey, M.W., 2013. Detecting api usage obstacles: a study of ios and android developer questions. In: Mining Software Repositories (MSR), 2013 10th IEEE Working Conference on, pp. 61–64. doi:10.1109/MSR.2013.6624006.
- Zhong, H., Xie, T., Zhang, L., Pei, J., Mei, H., 2009. Mapo: mining and recommending api usage patterns. In: Proceedings of the 23rd European Conference on ECOOP 2009 – Object-Oriented Programming. Springer-Verlag, Berlin, Heidelberg, pp. 318–343. doi:10.1007/978-3-642-03013-0_15.
- Zhong, H., Zhang, L., Mei, H., 2008. Inferring specifications of object oriented apis from api source code. In: 2008 15th Asia-Pacific Software Engineering Conference, pp. 221–228. doi:10.1109/APSEC.2008.54.